



Coverage of all Zeros in a Binary Matrix

Difficulty: **Easy**Accuracy: **55.68%**Submissions: **35K+**Points: **2**Average Time: **10m**

Given a binary matrix `mat[][]` containing only **0s** and **1s**, find the total coverage of all **0's**. The coverage of a particular 0 cell is defined by checking 1's in its four directions (left, right, up, or down). For each direction, if there is at least one 1 anywhere between the 0 and the boundary of the matrix, the coverage increases by one.



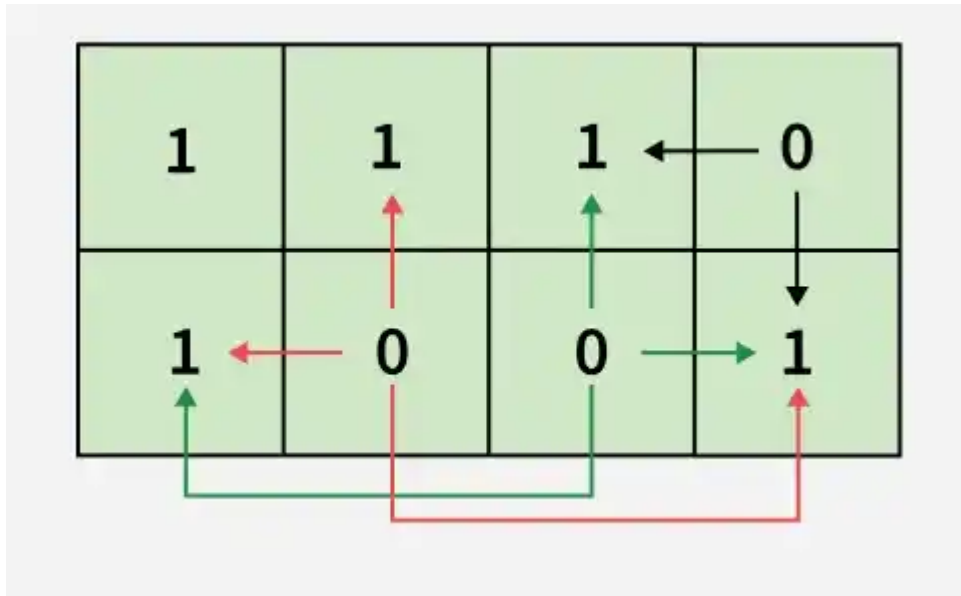
Return the sum of the coverage values for all 0 cells in the matrix.

Examples:

Input : `mat[][] = [[1, 1, 1, 0],`
`[1, 0, 0, 1]]`

Output : 8

Explanation: Coverage of first zero is 2. Coverages of other two zeros is 3 Total coverage = 2 + 3 + 3 = 8



Input: `mat[][] = [[0, 1, 0],`
`[0, 1, 1],`
`[0, 0, 0]]`

Output: 6

Explanation: Total Coverage is 1 + 2 + 1 + 0 + 1 + 1 = 6



Input: mat[][] = [[0, 1]]

Output: 1

Explanation: There are only 1 coverage. Therefore answer for this test case is 1.

Constraints:

$1 \leq \text{matrix.size}, \text{matrix}[0].\text{size} \leq 100$

[Try more examples](#)

Expected Complexities

Topic Tags

Related Articles

Python3

 Start Timer 

```
1 class Solution:
2     def findCoverage(self, mat):
3         n = len(mat)
4         m = len(mat[0])
5
6         left = [[0] * m for _ in range(n)]
7         right = [[0] * m for _ in range(n)]
8         up = [[0] * m for _ in range(n)]
9         down = [[0] * m for _ in range(n)]
10
11         # Left
12         for i in range(n):
13             seen = 0
14             for j in range(m):
15                 left[i][j] = seen
16                 if mat[i][j] == 1:
17                     seen = 1
18
19         # Right
20         for i in range(n):
21             seen = 0
22             for j in range(m - 1, -1, -1):
23                 right[i][j] = seen
24                 if mat[i][j] == 1:
25                     seen = 1
26
27         # Up
28         for j in range(m):
29             seen = 0
30             for i in range(n):
31                 up[i][j] = seen
32                 if mat[i][j] == 1:
33                     seen = 1
34
35         # Down
```

Menu



```
37         seen = 0
38         for i in range(n - 1, -1, -1):
39             down[i][j] = seen
40             if mat[i][j] == 1:
41                 seen = 1
42
43     ans = 0
44
45     for i in range(n):
46         for j in range(m):
47             if mat[i][j] == 0:
48                 ans += left[i][j] + right[i][j] + up[i][j] + down[i][j]
49
50     return ans
```

